

HPC Prac 1

1. Headers and Namespace

```
cpp
CopyEdit
#include <iostream>
#include <vector>
#include <unordered_map>
#include <unordered_set>
#include <queue>
#include <thread>
#include <future>
#include <atomic>
using namespace std;
```

- Includes necessary C++ standard library headers.
 - `iostream` - for input/output
 - `vector`, `unordered_map`, `unordered_set` - data structures
 - `queue` - for BFS
 - `thread`, `future` - for multithreading
 - `atomic` - for atomic variables (used for thread safety)
 - `using namespace std` to avoid prefixing `std::`
-

2. Graph Class Definition

```
cpp
CopyEdit
class Graph {
private:
    unordered_map<int, vector<int>> adjList;
```

- Graph represented using an adjacency list (`adjList`)
- Maps each node to a list of its neighbors

```
cpp
CopyEdit
public:
    void addEdge(int u, int v) {
        adjList[u].push_back(v);
        adjList[v].push_back(u);
    }
```

- Adds a bidirectional edge between nodes `u` and `v`

```
cpp
CopyEdit
    vector<int> getNeighbors(int u) {
        if (adjList.find(u) != adjList.end()) {
            return adjList[u];
        }
        return {};
    }
```

- Returns neighbors of node `u`, or an empty vector if node doesn't exist

```
cpp
CopyEdit
void printGraph() {
    cout << "\nGraph Structure:" << endl;
    for (auto& entry : adjList) {
        cout << entry.first << " -> ";
        for (int neighbor : entry.second) {
            cout << neighbor << " ";
        }
        cout << endl;
    }
};
```

- Prints the entire graph in adjacency list format
-

3. Parallel BFS

```
cpp
CopyEdit
vector<int> parallelBFS(Graph& graph, int start) {
    unordered_set<int> visited;
    queue<int> q;
    vector<int> bfsTraversal;
    atomic<bool> done(false);
```

- Initialize `visited` set, queue for BFS, and result vector
- `done` is declared but unused — can be removed

```
cpp
CopyEdit
    q.push(start);
    visited.insert(start);
```

- Start BFS from `start` node

```
cpp
CopyEdit
    while (!q.empty()) {
        int levelSize = q.size();
        vector<future<void>> futures;
```

- For each level in the BFS tree, store current size and prepare to launch threads

```
cpp
CopyEdit
        for (int i = 0; i < levelSize; i++) {
            int currentNode = q.front();
            q.pop();
            bfsTraversal.push_back(currentNode);
```

- Process each node in the current level

```

cpp
CopyEdit
    futures.push_back(async(launch::async, [&]() {
        for (int neighbor : graph.getNeighbors(currentNode)) {
            if (visited.find(neighbor) == visited.end()) {
                visited.insert(neighbor);
                q.push(neighbor);
            }
        }
    }));
}

```

- Each node's neighbors are processed asynchronously (but beware of thread safety issues here!)
- Shared data structures (`visited`, `q`) are not thread-safe

```

cpp
CopyEdit
    for (auto& fut : futures) {
        fut.get();
    }
    return bfsTraversal;
}

```

- Wait for all threads at this level to complete
- Return traversal result

⚠ Thread-safety warning: Modifying `queue` and `visited` from multiple threads without locking can lead to undefined behavior.

4. Recursive Parallel DFS

```

cpp
CopyEdit
void dfsRecursive(Graph& graph, int node, unordered_set<int>& visited,
vector<int>& dfsTraversal) {
    visited.insert(node);
    dfsTraversal.push_back(node);
}

```

- Standard recursive DFS, marking `node` as visited

```

cpp
CopyEdit
    vector<int> neighbors = graph.getNeighbors(node);
    vector<future<void>> futures;
    for (int neighbor : neighbors) {
        if (visited.find(neighbor) == visited.end()) {
            futures.push_back(async(launch::async, [&]() {
                dfsRecursive(graph, neighbor, visited, dfsTraversal);
            }));
        }
    }
}

```

- Launch DFS calls to unvisited neighbors in parallel

```
cpp
CopyEdit
    for (auto& fut : futures) {
        fut.get();
    }
}
```

- Wait for all recursive threads to finish

⚠ Again, this is **not thread-safe**, as both `visited` and `dfsTraversal` are accessed from multiple threads.

5. Wrapper for Parallel DFS

```
cpp
CopyEdit
vector<int> parallelDFS(Graph& graph, int start) {
    unordered_set<int> visited;
    vector<int> dfsTraversal;
    dfsRecursive(graph, start, visited, dfsTraversal);
    return dfsTraversal;
}
```

- Calls the recursive function and returns the traversal
-

6. Print Traversal Output

```
cpp
CopyEdit
void printTraversalAsTree(const vector<int>& traversal, const string&
traversalType) {
    cout << "\n" << traversalType << " Traversal Output: " << endl;
    for (int node : traversal) {
        cout << node << " ";
    }
    cout << endl;
}
```

- Utility function to print BFS/DFS output
-

7. Main Function

```
cpp
CopyEdit
int main() {
    Graph graph;
    int numEdges;
```

```

    cout << "Enter the number of edges you want to add to the graph:" <<
endl;
    cin >> numEdges;

```

- Prompt user for number of edges

```

cpp
CopyEdit
    cout << "Enter edges as pairs of nodes (e.g., 0 1 for an edge between 0
and 1): " << endl;
    for (int i = 0; i < numEdges; i++) {
        int u, v;
        cin >> u >> v;
        graph.addEdge(u, v);
    }
    graph.printGraph();

```

- Add each edge and print the final graph

```

cpp
CopyEdit
    bool continueRunning = true;
    while (continueRunning) {
        cout << "\nChoose an option: " << endl;
        cout << "1. Parallel BFS" << endl;
        cout << "2. Parallel DFS" << endl;
        cout << "3. Exit" << endl;
        cout << "Enter your choice: ";
        int choice;
        cin >> choice;
    }

```

- Menu loop: user chooses BFS, DFS, or exit

```

cpp
CopyEdit
    switch (choice) {
        case 1: {
            cout << "Enter starting node for BFS: ";
            int startBFS;
            cin >> startBFS;
            cout << "\nRunning Parallel BFS..." << endl;
            vector<int> bfsTraversal = parallelBFS(graph, startBFS);
            printTraversalAsTree(bfsTraversal, "BFS");
            break;
        }
    }

```

- Handle Parallel BFS

```

cpp
CopyEdit
        case 2: {
            cout << "Enter starting node for DFS: ";
            int startDFS;
            cin >> startDFS;
            cout << "\nRunning Parallel DFS..." << endl;
            vector<int> dfsTraversal = parallelDFS(graph, startDFS);
            printTraversalAsTree(dfsTraversal, "DFS");
            break;
        }
    }

```

```
}
```

- Handle Parallel DFS

cpp

CopyEdit

```
case 3:
    continueRunning = false;
    cout << "Exited the program successfully" << endl;
    break;
```

- Exit program

cpp

CopyEdit

```
default:
    cout << "Invalid choice! Please choose a valid option." <<
endl;
    break;
}
```

- Handle invalid input

cpp

CopyEdit

```
if (continueRunning) {
    cout << "\nDo you want to continue? (yes/no): ";
    string userChoice;
    cin >> userChoice;
    if (userChoice == "no" || userChoice == "No") {
        continueRunning = false;
        cout << "Exited the program successfully" << endl;
    }
}
}
return 0;
}
```

- Ask user whether they want to repeat the process

☒ Summary

- Implements an **undirected graph**
- Supports **Parallel BFS** and **Parallel DFS** using `std::async`
- Simple menu-driven CLI for interaction